

Assignment 6

Date _____

Page _____

Q.1. p.s.v.m.c) {
int n = 3;
fun(3);

3
n

```
void fun (int n) {
    if (n == 0) { 3 == 0 x
        return;
    }
    s.o. print(n + " ");
    fun(n-1);
    s.o. print((n+1) + " ");
}
```

F(0) — pop.
F(1) — pop.
F(2) — pop.
F(3) — pop.
call stack

• call: fun (3):

output :- 3 2 1 2 3 4

(2) void fun (int n) {
if (n == 0) { 2 == 0 x
return;
}
s.o. print(n + " ");
fun (n-1);
s.o. print((n+1) + " ");
}

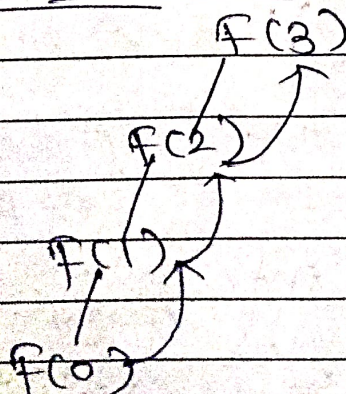
void fun (int n) {
if (n == 0) { 1 == 0 x
return;
}

s.o. print(n + " ");
fun(n-1);

s.o. print((n+1) + " ");

(4) void fun (int n) {
if (n == 0) { 0 == 0
return;
}
s.o. print(n + " ");
fun(n-1);
s.o. print((n+1) + " ");
}

Recursion tree



output:- x x y y x y

Q.2
func2

p.s.v.m.c() {
int n = 2;

2
n

fun(n);

void fun(int n) {

if (n == 0) {
return;

}

printf("x");

fun(n-1);

printf("y");

fun(n-1);

}

pop -> f(0)

pop -> f(0)

pop -> f(1)

pop -> f(0)

pop -> f(0)

pop -> f(1)

pop -> f(2)

all stack

5 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

fun(n-1);

printf("y");

fun(n-1);

6 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

fun(n-1);

printf("y");

fun(n-1);

7 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

fun(n-1);

2 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

fun(n-1);

printf("y");

fun(n-1);

}

3 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

fun(n-1);

printf("y");

fun(n-1);

}

4 void fun(int n) {

if (n == 0) {
return;

return;

printf("x");

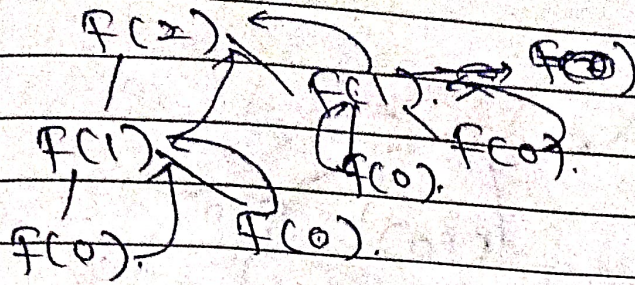
fun(n-1);

printf("y");

fun(n-1);

}

Recursive tree.



Q-3. p.s.v.m.c()

int n=4;

4
n

Fun(n);

```

void Fun(int n) {
    if (n <= 1) {
        so print (n + " ");
        return;
    }
    Fun(n-2);
    so print (n + " ");
    return;
}
  
```

Fun(n-2);

so print (n + " ");

Fun(n-2);

void Fun(int n) {

if (n <= 1) {

so print (n + " ");

return;

Fun(n-2);

so print (n + " ");

Fun(n-2);

void Fun(int n) {

if (n <= 1) {

so print (n + " ");

return;

Fun(n-2);

so print (n + " ");

Fun(n-2);

pop: F(0)

pop: F(0)

pop: F(2)

pop: F(0)

pop: F(0)

pop: F(2)

pop: F(4)

output:-

2 0 4 0 2

0

call stack 2

void Fun(int n) {

if (n <= 1) {

so print (n + " ");

return;

Fun(n-2);

so print (n + " ");

Fun(n-2);

void Fun(int n) {

if (n <= 1) {

so print (n + " ");

return;

Fun(n-2);

so print (n + " ");

Fun(n-2);

void Fun(int n) {

if (n <= 1) {

so print (n + " ");

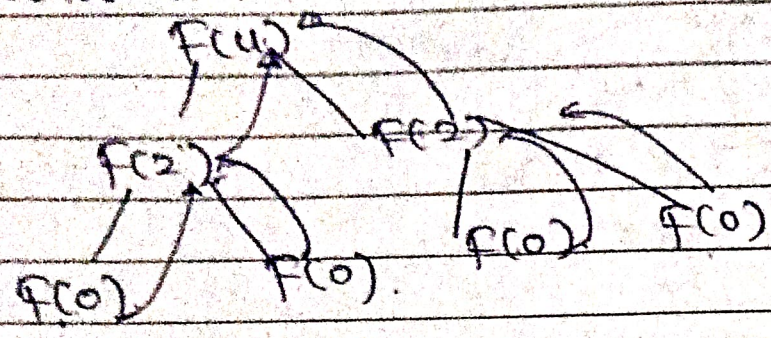
return;

Fun(n-2);

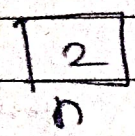
so print (n + " ");

Fun(n-2);

Recursion tree.

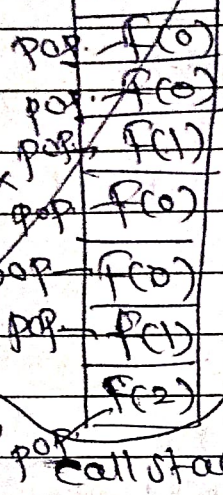


Q.4. psvm()?
int n = 2;
fun(n);
1.4.1



```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```



```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```

```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```

```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```

```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```

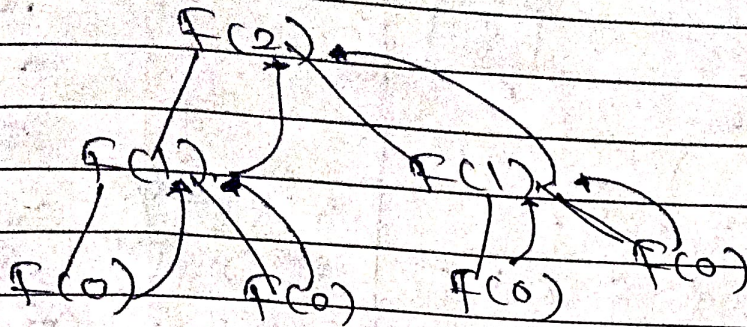
```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```

```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    printf("%d\n", n);
    fun(n-1);
}
  
```


Recursion tree



1-2 = -1

3

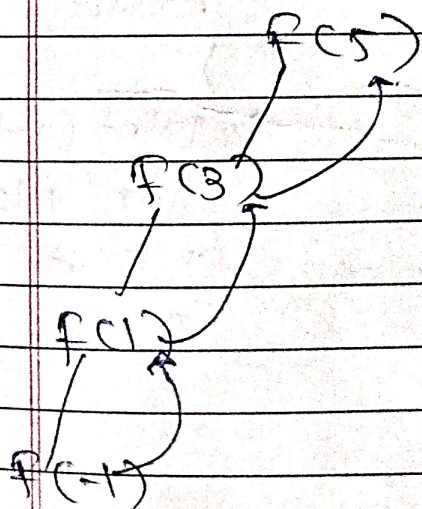
Q.5. P.S.V.m() {
int n=5; 5
Fun(n);
}

3 1 1
3 5.

```

void Fun(int n) {
    if (n <= 0) { return; }
    So print (n + " ");
    Fun(n-2);
    So print (n + " ");
}
  
```

Recursion tree.



2) void Fun(int n) {
if (n <= 0) {
return; }
So print (n + " ");
Fun(n-2);
So print (n + " ");
}

3) void Fun(int n) {
if (n <= 0) {
return; }
So print (n + " ");
Fun(n-2);
So print (n + " ");
}

4) void Fun(int n) {
-1 <= 0. if (n <= 0) {
return; }
So print (n + " ");
Fun(n-2);
So print (n + " ");
}

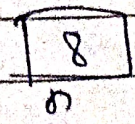
call stack

F(1) - pop
F(1) - pop
F(3) - pop
F(5) - pop

output: 8 4 2 1 2 4 8

Q.6.

```
p.s.v.m(){
    int n = 8;
```



Fun(n);

```
{
    void Fun(int n){
        if(n <= 1){ 8 <= 1 X
            so.print(n + " ");
            return;
        }
        so.print(n + " ");
        Fun(n/2);
        so.print(n + " ");
    }
}
```

F(1) - pop.
F(2) - pop.
F(4) - pop.
F(8) - pop.

call stack

```
2) void Fun(int n){
    if(n <= 1){ 8 <= 1 X
        so.print(n + " ");
        return;
    }
    so.print(n + " ");
    Fun(n/2);
}
```

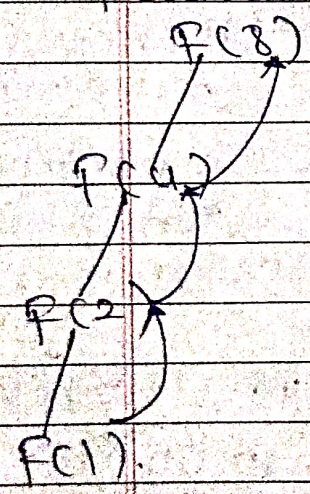
```
3) void Fun(int n){
    if(n <= 1){ 8 <= 1 X
        so.print(n + " ");
        return;
    }
    so.print(n + " ");
    Fun(n/2);
}
```

```
so.print(n + " ");
Fun(n/2);
so.print(n + " ");
```

```
4) void Fun(int n){
    if(n <= 1){ 8 <= 1 X
        so.print(n + " ");
        return;
    }
    so.print(n + " ");
    Fun(n/2);
}
```

```
so.print(n + " ");
Fun(n/2);
so.print(n + " ");
```

Recursion tree.



output:- A 3 B A 2 A 1 B 2 B 4 B 6.

Date _____

Page _____

Q.7). void Fun()

{

int n = 3;

Fun(n);

{

void Fun(int n)

{ if (n <= 0) return;

return;

}

do: print("A " + n + " ");

Fun(n-1);

do: print("B " + (n*2) + " ");

}

2) void Fun(int n)

{ if (n <= 0) return;

return;

{

do: print("A " + n + " ");

Fun(n-1);

do: print("B " + (n*2) + " ");

}

3) void Fun(int n)

{ if (n <= 0) return;

return;

{ do: print("A " + n + " ");

Fun(n-1);

do: print("B " + (n*2) + " ");

}

4) void Fun(int n)

{ if (n <= 0) return;

return;

{ do: print("A " + n + " ");

Fun(n-1);

do: print("B " + (n*2) + " ");

F(0) — pop.

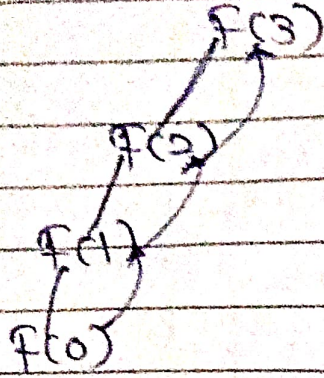
F(1) — pop.

F(2) — pop.

F(3) — pop.

call stack

Recursion tree.



Q. 8. p.s.v.m ()

output. int n=4;

1. fun (n);

void fun (int n){

if (n==0){ u=0x

return;

fun (n-1);

So print (n + " ");

fun (n-2);

```

1 5                                     3
2 ③ void fun(int n){
3     if (n == 0){ 3 == 0x
4         return;

```

```

    } Fun(n-1);
    S.c.print(n + " ");
    Fun(n-2);
}

```

```

3 void fun (int n) {
    if (n == 0) { 2 == 0 }
        return;
    fun (n-1);
    printf (" ");
    fun (n-2);
}

```

$f(-4)$	
$f(-3)$	
$f(-2)$	
$f(-1)$	
$f(0)$	pop.
$f(1)$	
$f(2)$	
$f(3)$	
$f(4)$	

Call Stock

```

5 void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    sum += n;
    fun(n-2);
}

```

```

void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n-1);
    cout << n << " ";
    fun(n-2);
}

```


① void Fun(int n) {
⁻¹
 if (n == 0) { -1 == 0X
 }
 return;
 Fun(n-1);
 So print (n + " ");
 Fun(n-2);
 }

② void Fun(int n) {
⁻²
 if (n == 0) { -2 == 0X
 }
 return;
 Fun(n-1);
 So print (n + " ");
 Fun(n-2);
 }

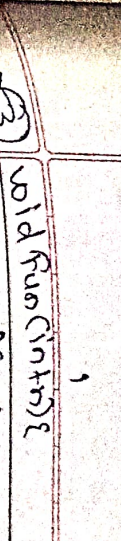
③ void Fun(int n) {
⁻³
 if (n == 0) { -3 == 0X
 }
 return;
 Fun(n-1);
 So print (n + " ");
 Fun(n-2);
 }

④ void Fun(int n) {
⁻⁴
 if (n == 0) { -4 == 0X
 }
 return;
 Fun(n-1);
 So print (n + " ");
 Fun(n-2);
 }

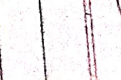
Q.9. p.s.w.m() {
int n = 3;
return n;
}



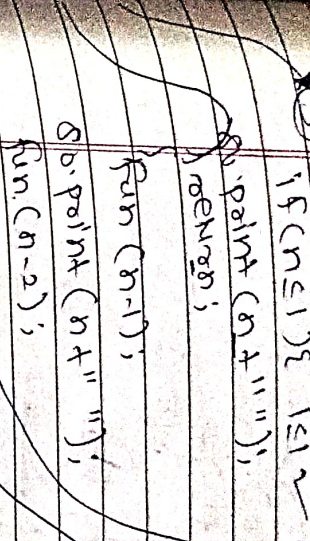
(2) void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



(3) void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



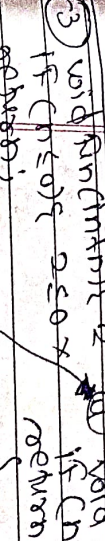
fun(n-1) {
do point (n + " ");
fun(n-2);
}



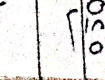
fun(n-2) {
do point (n + " ");
fun(n-3);
}



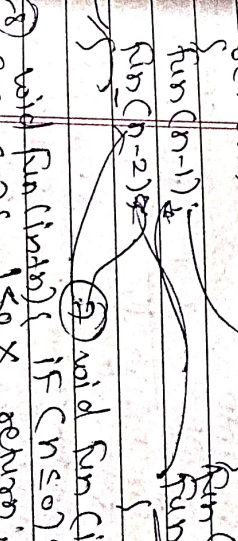
void p.s.w.m() {
int n = 4;
fun(n);
}



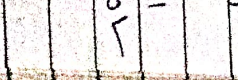
void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



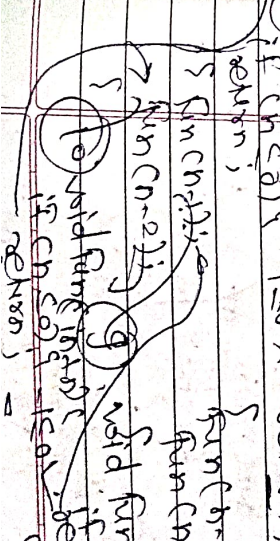
void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



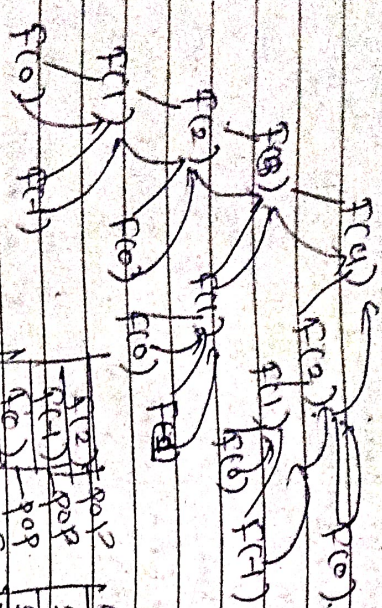
void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



void fun(int n) {
if (n <= 1) {
do point (n + " ");
return;
}
fun(n-1);
do point (n + " ");
fun(n-2);
}



Recursion tree.



Q.8. p.s.v.m.c.i
int n = 4;

```

void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}
  
```

f(4)	pop	f(4)	pop
f(3)	pop	f(3)	pop
f(2)	pop	f(2)	pop
f(1)	pop	f(1)	pop
f(0)	pop	f(0)	pop

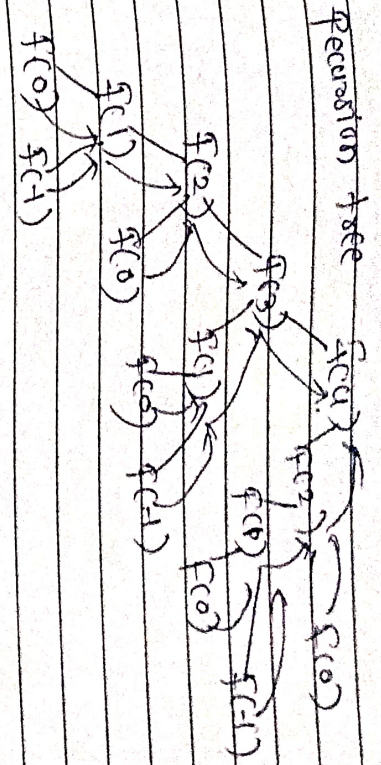
```

(1) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}

(2) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}

(3) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}
  
```

Recursion tree



```

(8) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}
  
```

```

(9) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}

(10) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}

(11) void fun(int n) {
    if (n <= 0) {
        // action;
        return;
    }
    // action;
    fun(n-1);
    // action;
    fun(n+1);
}
  
```